
CS-564: Foundations of ML

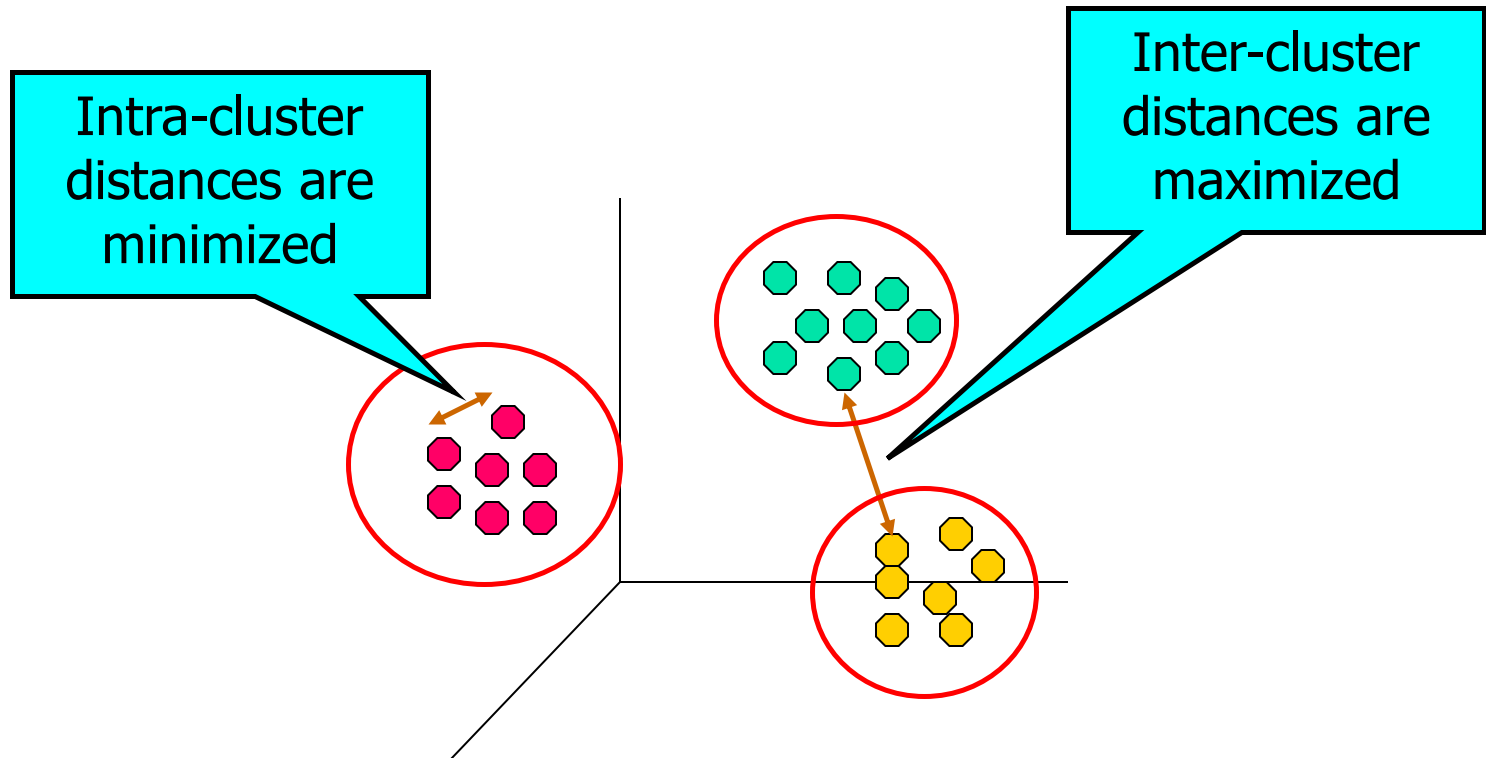
Clustering

Outline

- Cluster Analysis: Basic Concepts 
- Partitioning Methods
- Hierarchical Methods
- Density-Based Methods
- Grid-Based Methods
- Evaluation of Clustering
- Summary

What is Cluster Analysis?

- Finding groups of objects such that the objects in a group will be similar (or related) to one another and different from (or unrelated to) the objects in other groups



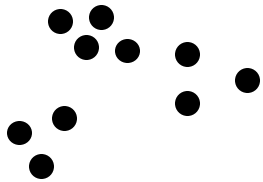
What is Cluster Analysis?

- Cluster analysis (or *clustering*, *data segmentation*, ...)
 - Finding similarities between data according to the characteristics found in the data and grouping similar data objects into clusters
- **Unsupervised learning**: no predefined classes (i.e., *learning by observations* vs. learning by examples: supervised)
- Typical applications
 - As a **stand-alone tool** to get insight into data distribution
 - As a **preprocessing step** for other algorithms

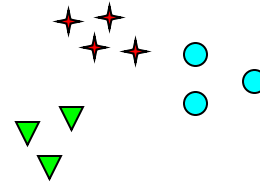
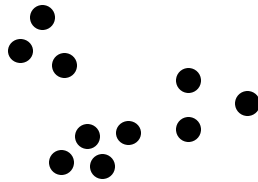
What is not Cluster Analysis?

- Supervised classification
 - Have class label information
- Simple segmentation
 - Dividing students into different registration groups alphabetically, by last name
- Results of a query
 - Groupings are a result of an external specification
- Graph partitioning
 - Some mutual relevance and synergy, but areas are not identical

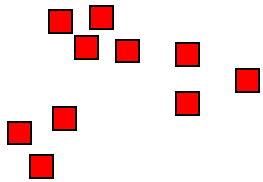
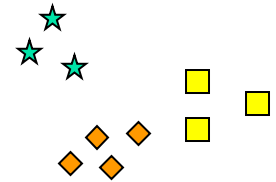
Notion of a cluster can be ambiguous



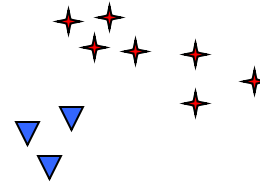
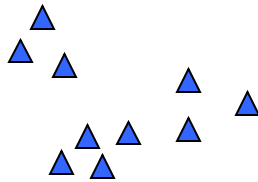
How many clusters?



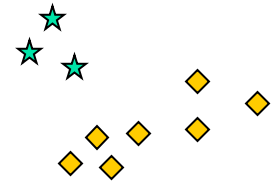
Six Clusters



Two Clusters



Four Clusters



Quality: What Is Good Clustering?

- A good clustering method will produce high quality clusters
 - high intra-class similarity: **cohesive** within clusters
 - low inter-class similarity: **distinctive** between clusters
- Quality of a clustering method depends on
 - the similarity measure used by the method
 - its implementation, and
 - its ability to discover some or all of the hidden patterns

Measure the Quality of Clustering

■ Dissimilarity/Similarity metric

- Similarity is expressed in terms of a distance function, typically metric: $d(i, j)$
- Definitions of **distance functions** are usually rather different for interval-scaled, boolean, categorical, ordinal ratio, and vector variables
- Weights should be associated with different variables based on applications and data semantics

■ Quality of clustering

- There is usually a separate “quality” function that measures the “goodness” of a cluster
- It is hard to define “similar enough” or “good enough”
 - Answer is typically highly subjective

Considerations for Cluster Analysis

- Partitioning criteria
 - Single level vs. hierarchical partitioning (often, multi-level hierarchical partitioning is desirable)
- Separation of clusters
 - Exclusive (e.g., one customer belongs to only one region) vs. non-exclusive (e.g., one document may belong to more than one class)
- Similarity measure
 - Distance-based (e.g., Euclidean, road network, vector) vs. connectivity-based (e.g., density or contiguity)
- Clustering space
 - Full space (often when low dimensional) vs. subspaces (often in high-dimensional clustering)

Major Clustering Approaches (I)

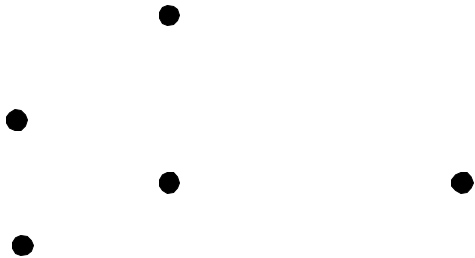
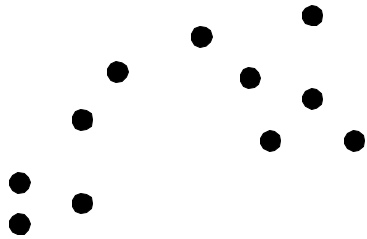
- Partitioning approach:
 - Construct various partitions and then evaluate them by some criterion, e.g., minimizing the sum of square errors
 - Typical methods: k-means, k-medoids, CLARANS
- Hierarchical approach:
 - Create a hierarchical decomposition of the set of data (or objects) using some criterion
 - Typical methods: Diana, Agnes, BIRCH, CAMELEON
- Density-based approach:
 - Based on connectivity and density functions
 - Typical methods: DBSACN, OPTICS, DenClue
- Grid-based approach:
 - Based on a multiple-level granularity structure
 - Typical methods: STING, WaveCluster, CLIQUE

Major Clustering Approaches (II)

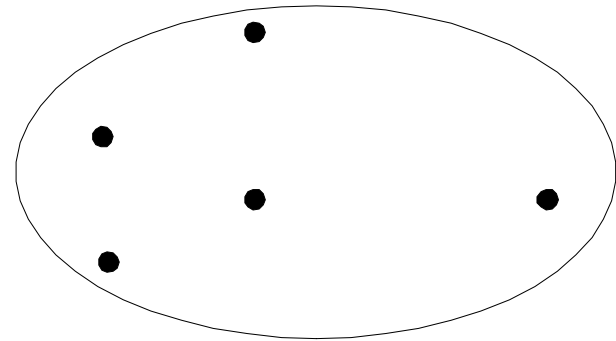
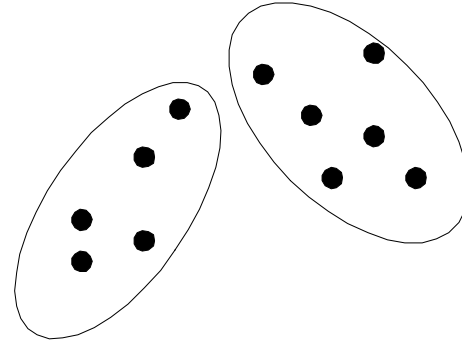
- Model-based:
 - A model is hypothesized for each of the clusters and tries to find the best fit of that model to each other
 - Typical methods: EM, COBWEB
- Frequent pattern-based:
 - Based on the analysis of frequent patterns
 - Typical methods: p-Cluster
- User-guided or constraint-based:
 - Clustering by considering user-specified or application-specific constraints
 - Typical methods: COD (obstacles), constrained clustering
- Link-based clustering:
 - Objects are often linked together in various ways
 - Massive links can be used to cluster objects: SimRank, LinkClus

With Some Examples

Partitional Clustering

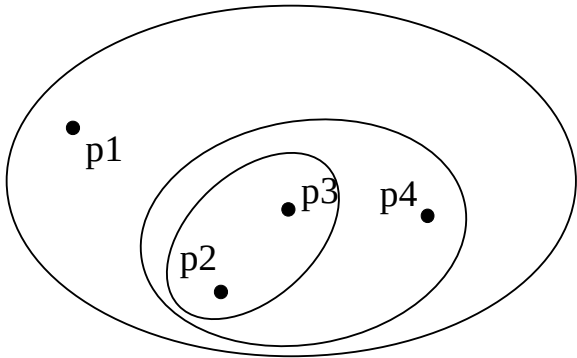


Original Points

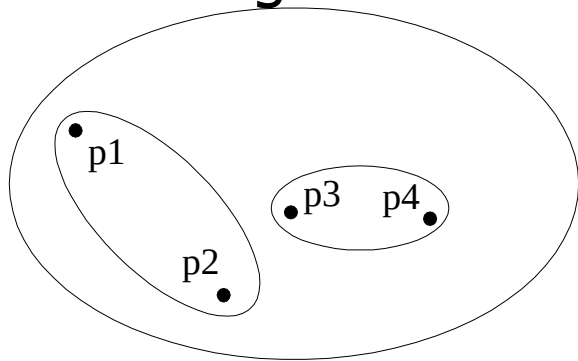


A Partitional Clustering

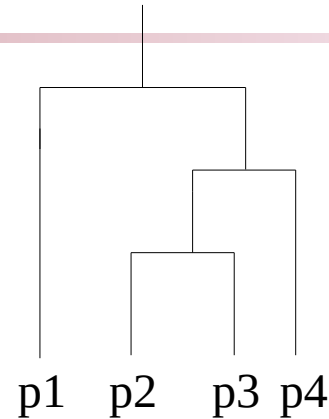
Hierarchical Clustering



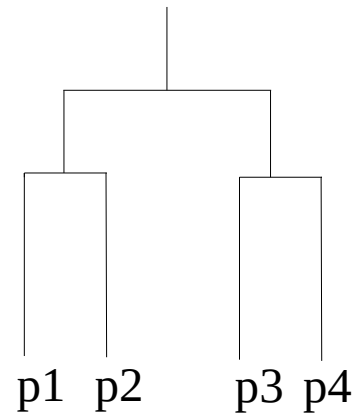
Traditional Hierarchical Clustering



Non-traditional Hierarchical Clustering



Traditional Dendrogram



Non-traditional Dendrogram

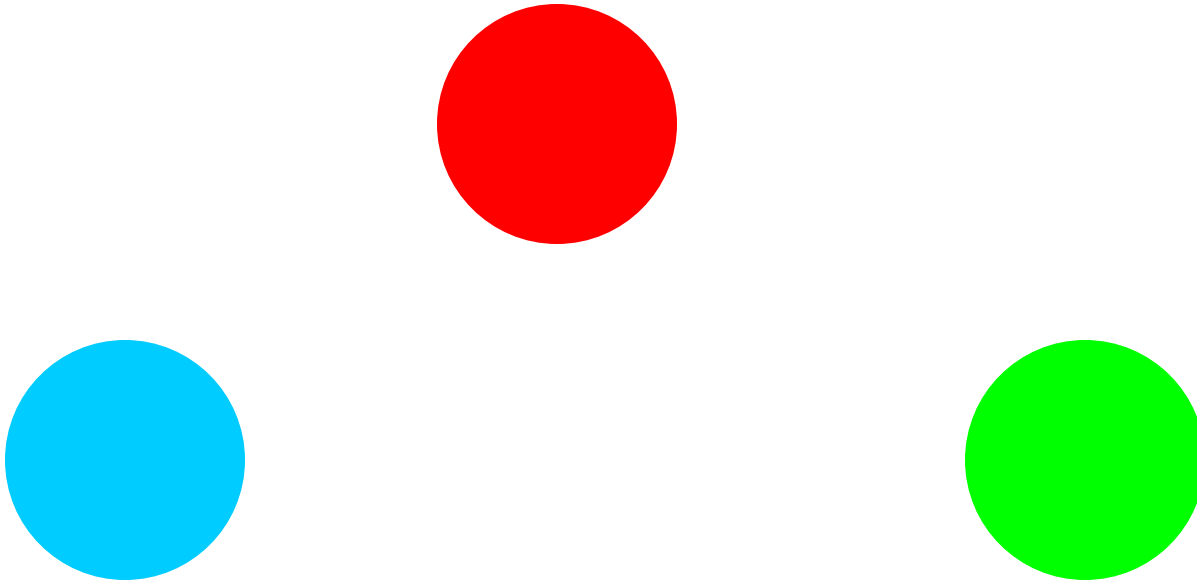
Other distinctions between sets of clusters

- **Exclusive versus non-exclusive**
 - In non-exclusive clusterings, points may belong to multiple clusters
 - Can represent multiple classes or 'border' points
- **Fuzzy versus non-fuzzy**
 - In fuzzy clustering, a point belongs to every cluster with some weight between 0 and 1
 - Weights must sum to 1
 - Probabilistic clustering has similar characteristics
- **Partial versus complete**
 - In some cases, we only want to cluster some of the data
- **Heterogeneous versus homogeneous**
 - Cluster of widely different sizes, shapes, and densities

Types of Clusters: Well-Separated

- Well-Separated Clusters:

- A cluster is a set of points such that any point in a cluster is closer (or more similar) to every other point in the cluster than to any point not in the cluster

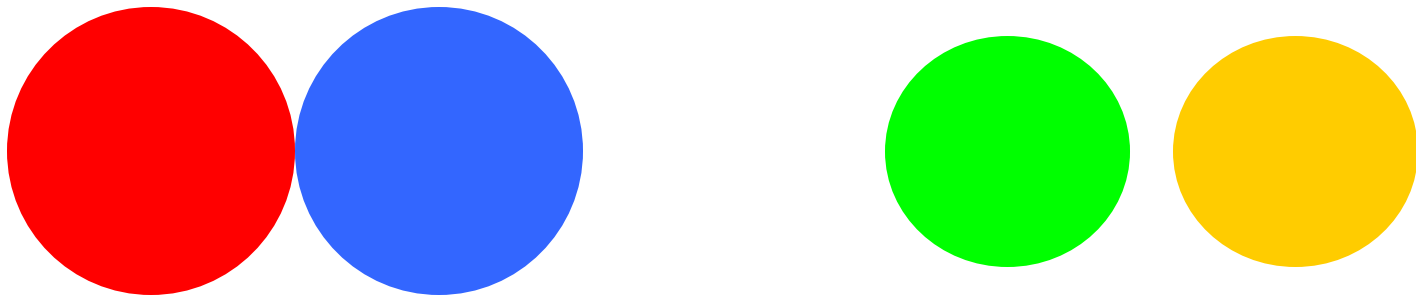


3 well-separated clusters

Types of Clusters: Center-Based

■ Center-based

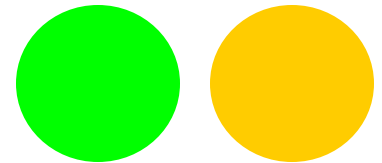
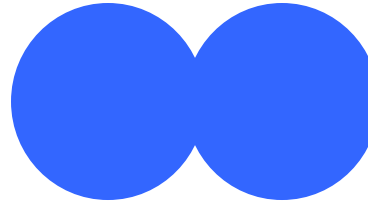
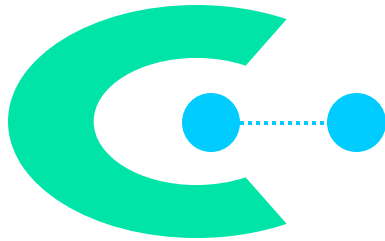
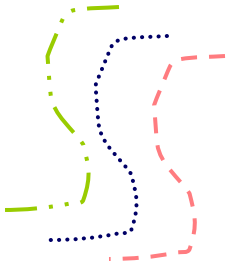
- A cluster is a set of objects such that an object in a cluster is closer (more similar) to the “center” of a cluster, than to the center of any other cluster
- The center of a cluster is often a **centroid**, the average of all the points in the cluster, or a **medoid**, the most “representative” point of a cluster



4 center-based clusters

Types of Clusters: Contiguity-Based

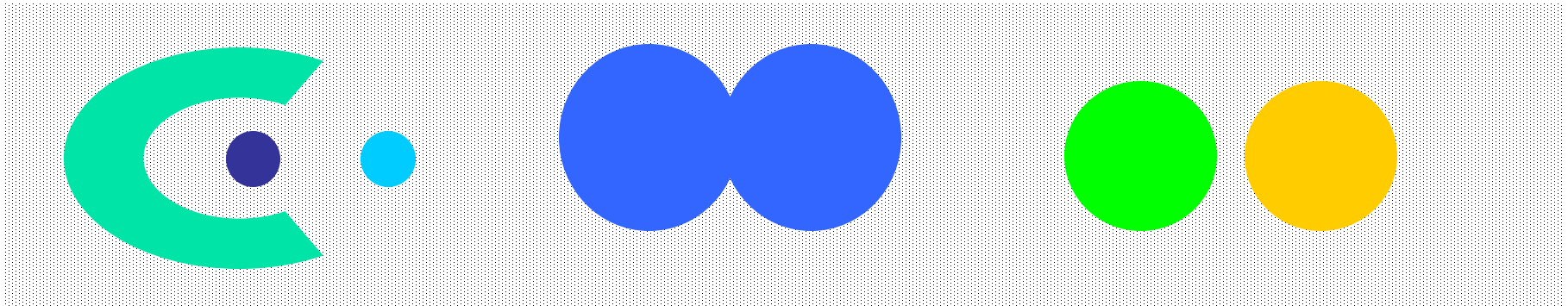
- Contiguous Cluster (Nearest neighbor or Transitive)
 - A cluster is a set of points such that a point in a cluster is closer (or more similar) to one or more other points in the cluster than to any point not in the cluster



8 contiguous clusters

Types of Clusters: Density-Based

- Density-based
 - A cluster is a dense region of points, which is separated by low-density regions, from other regions of high density
 - Used when the clusters are irregular or intertwined, and when noise and outliers are present

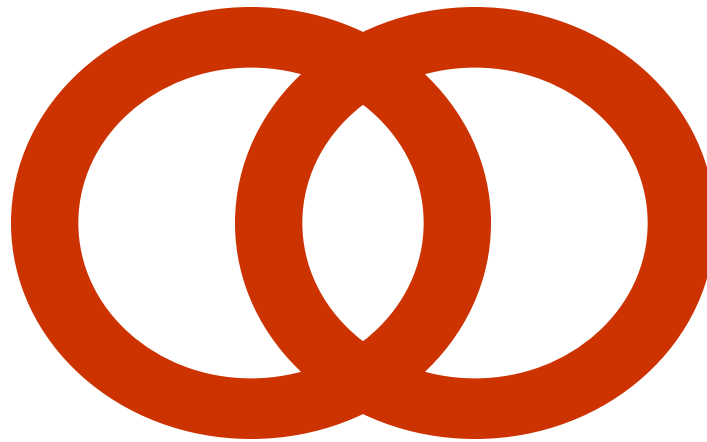


6 density-based clusters

Types of Clusters: Conceptual Clusters

- Shared Property or Conceptual Clusters
 - Finds clusters that share some common property or represent a particular concept

.



2 Overlapping Circles

Partitioning Algorithms: Basic Concept

- Partitioning method: Partitioning a database D of n objects into a set of k clusters, such that the sum of squared distances is minimized (where c_i is the centroid or medoid of cluster C_i)

$$E = \sum_{i=1}^k \sum_{p \in C_i} (d(p, c_i))^2$$

- Given k , find a partition of k clusters that optimizes the chosen partitioning criterion
 - Global optimal: exhaustively enumerate all partitions
 - Heuristic methods: *k-means* and *k-medoids* algorithms
 - k-means (MacQueen'67, Lloyd'57/'82): Each cluster is represented by the center of the cluster
 - k-medoids or PAM (Partition around medoids) (Kaufman & Rousseeuw'87): Each cluster is represented by one of the objects in the cluster

K-means Clustering

- Partitional clustering approach
- Each cluster is associated with a **centroid** (center point)
- Each point is assigned to the cluster with the closest centroid
- Number of clusters, K , must be specified
- Very simple but efficient approach

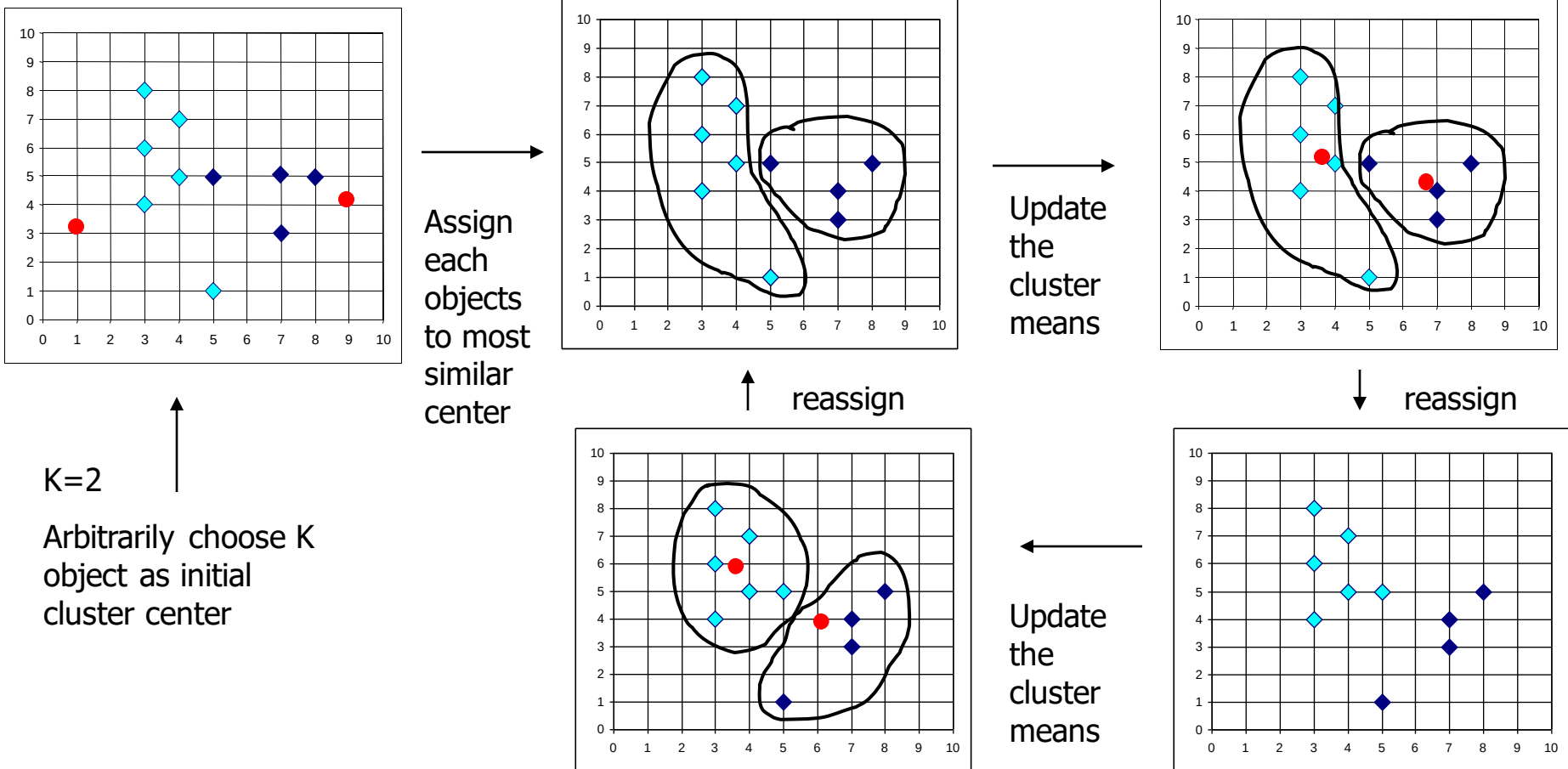
-
- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change
-

K-means Clustering – Details

- Initial centroids are often chosen randomly
 - Clusters produced vary from one run to another
- The centroid is (typically) the mean of the points in the cluster
- ‘Closeness’ is measured by Euclidean distance, cosine similarity, correlation, etc
- K-means will converge for common similarity measures mentioned above
- Most of the convergence happens in the first few iterations
 - Often the stopping condition is changed to ‘Until relatively few points change clusters’
- Time Complexity is $O(n * K * I * d)$
 - n = number of points, K = number of clusters, I = number of iterations, d = number of attributes
- **Space complexity** is $O((n+K)m)$, m -number of attributes

The *K*-Means Clustering Method

■ Example



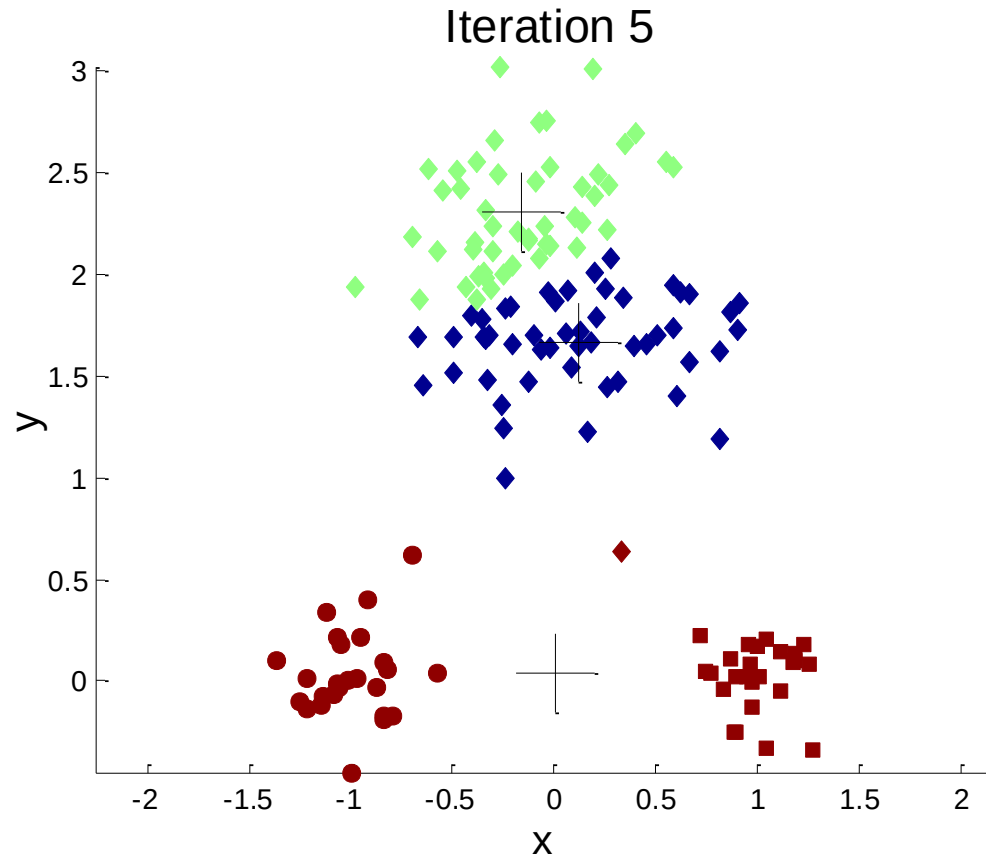
Evaluating K-means Clusters

- Most common measure is Sum of Squared Error (SSE)
 - For each point, the error is the distance to the nearest cluster
 - To get SSE, we square these errors and sum them.

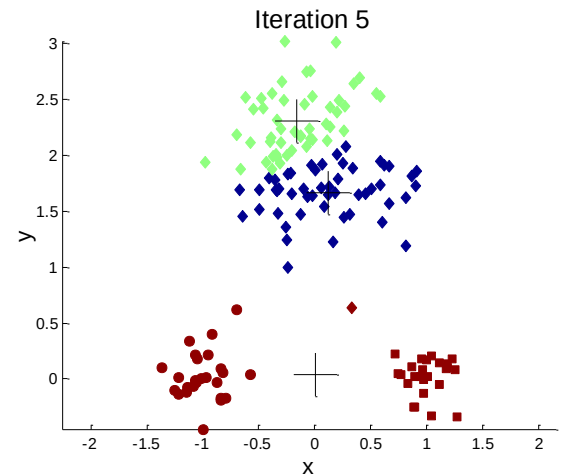
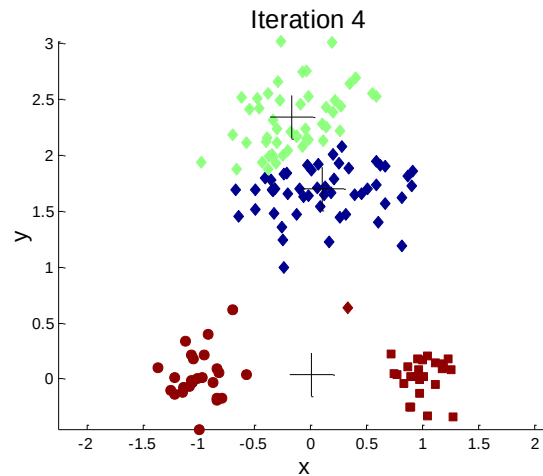
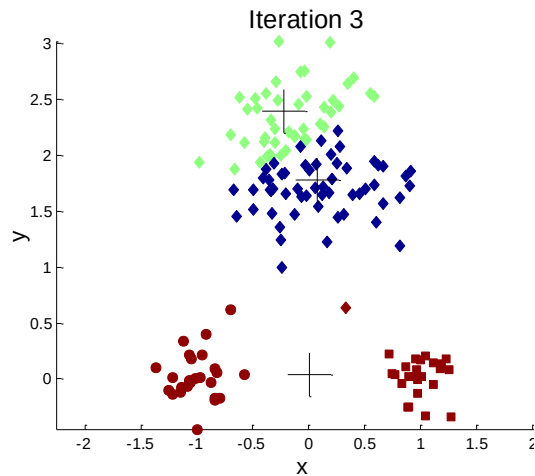
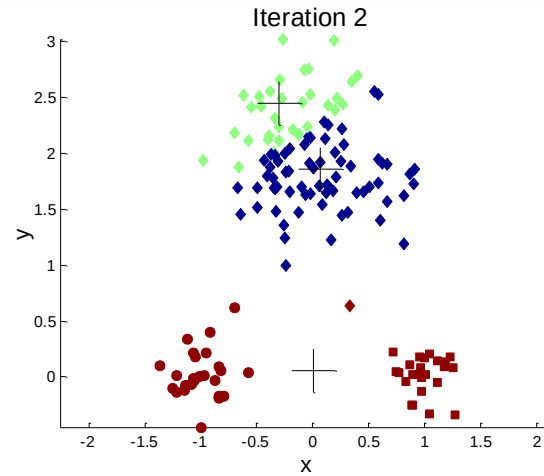
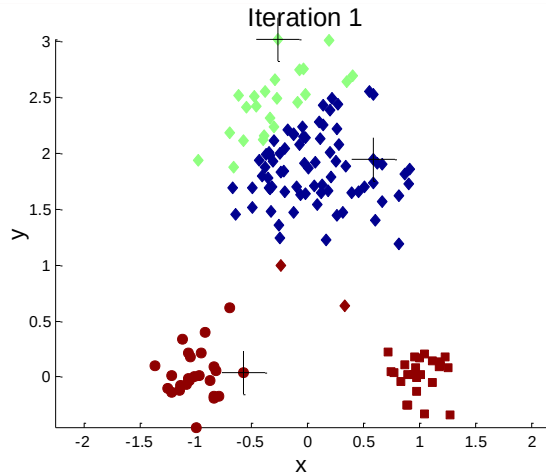
$$SSE = \sum_{i=1}^K \sum_{x \in C_i} dist^2(m_i, x)$$

- x is a data point in cluster C_i and m_i is the representative point for cluster C_i
 - can show that m_i corresponds to the center (mean) of the cluster
- Given two sets of clusters, we can choose the one with the smallest error
- One easy way to reduce SSE is to increase K , the number of clusters
 - A good clustering with smaller K can have a lower SSE than a poor clustering with higher K

Importance of Choosing Initial Centroids ...



Importance of Choosing Initial Centroids ...



Problems with Selecting Initial Points

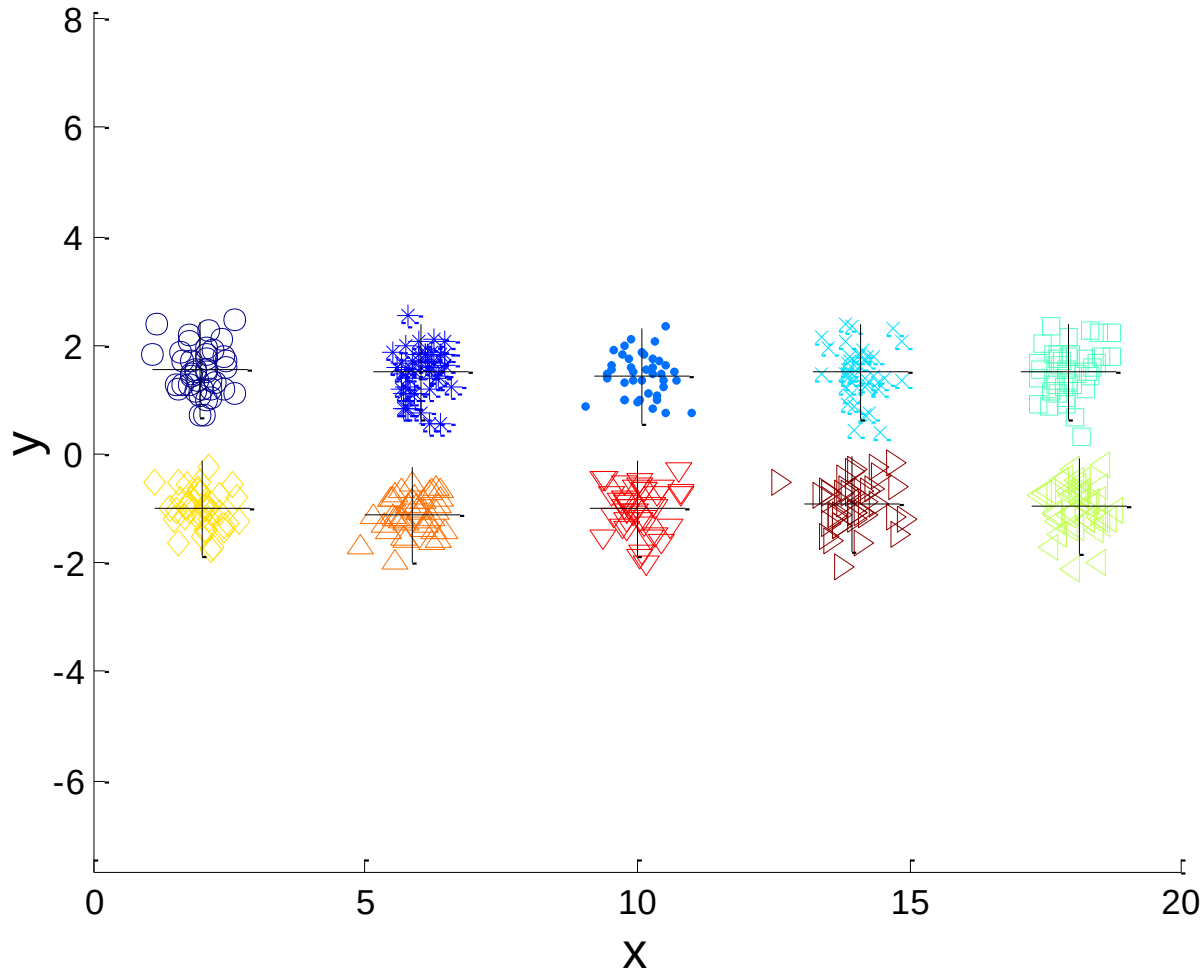
- If there are K 'real' clusters then the chance of selecting one centroid from each cluster is small
 - Chance is relatively small when K is large
 - If clusters are the same size, n , then

$$P = \frac{\text{number of ways to select one centroid from each cluster}}{\text{number of ways to select } K \text{ centroids}} = \frac{K!n^K}{(Kn)^K} = \frac{K!}{K^K}$$

- For example, if $K = 10$, then probability = $10!/10^{10} = 0.00036$
- Sometimes the initial centroids will readjust themselves in 'right' way, and sometimes they don't
- Consider an example of five pairs of clusters

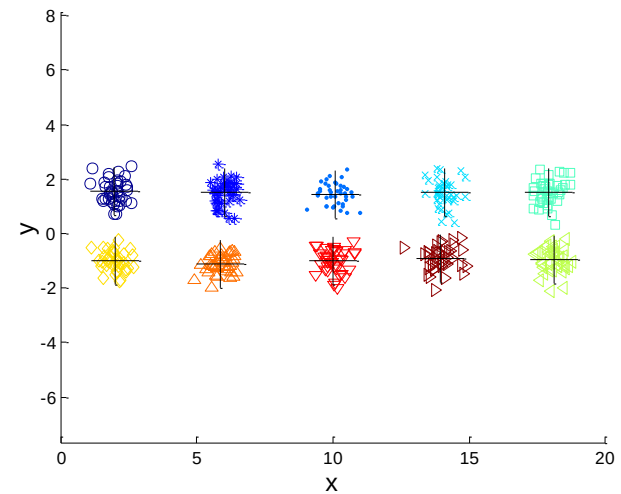
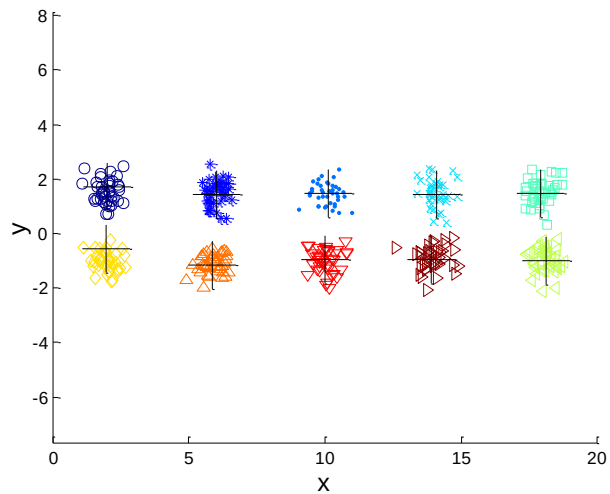
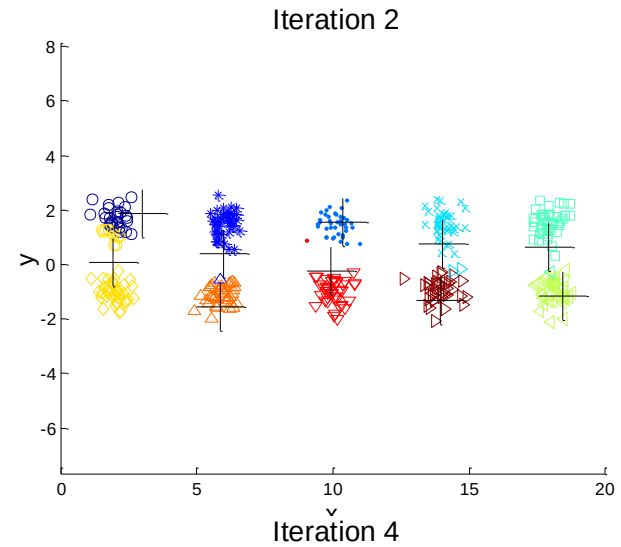
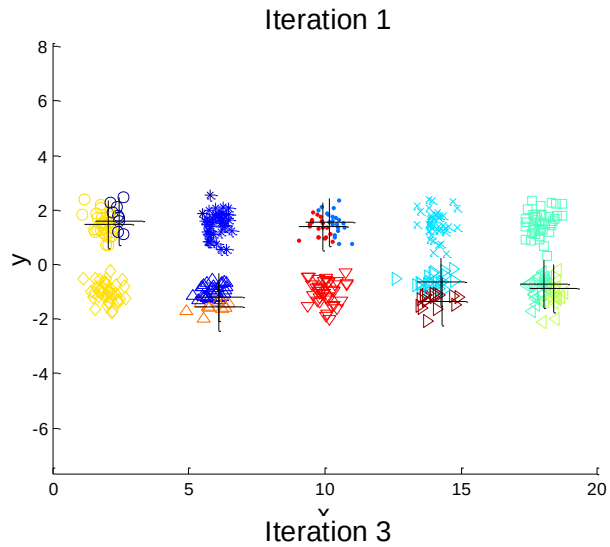
10 Clusters Example

Iteration 4



Starting with two initial centroids in one cluster of each pair of clusters

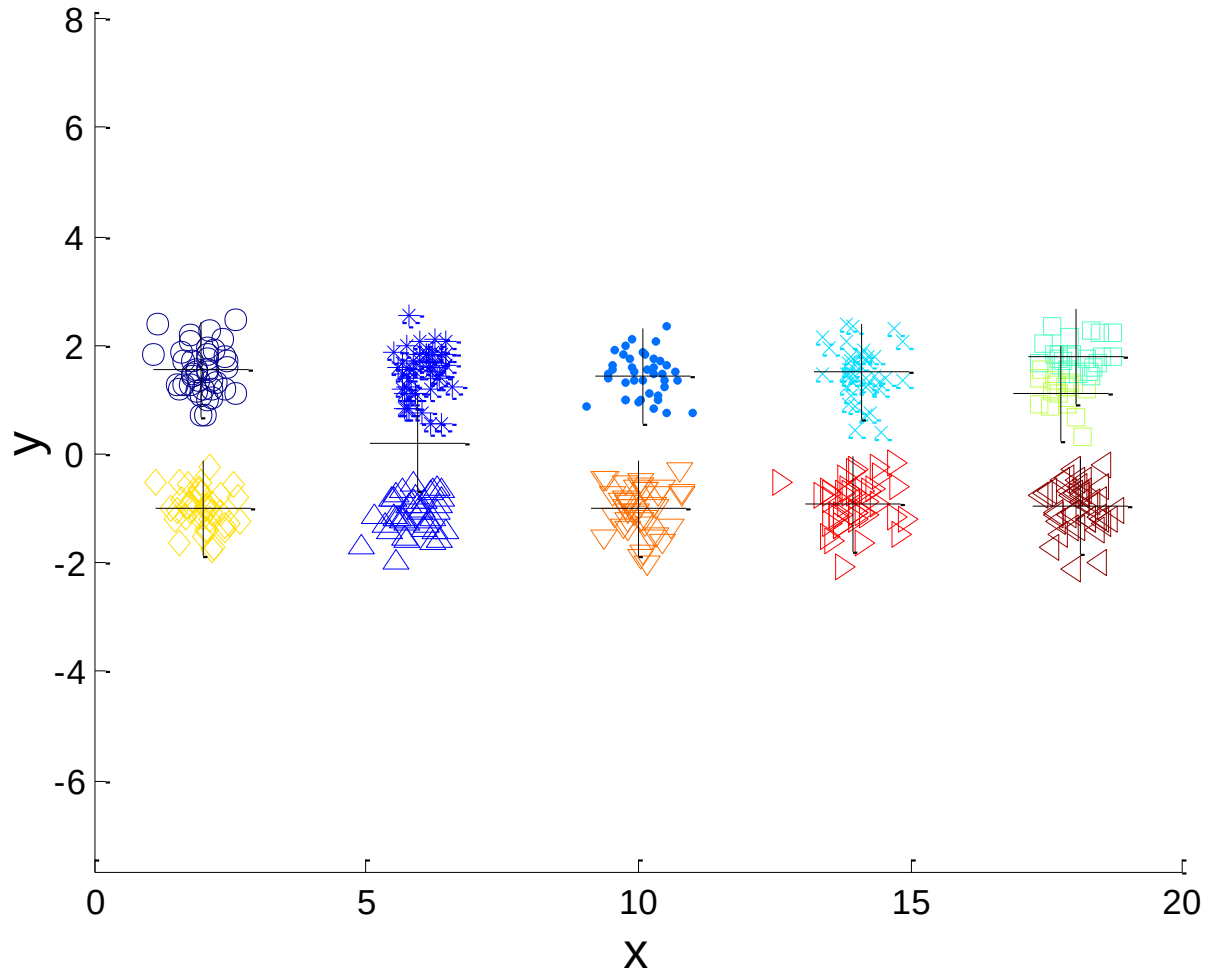
10 Clusters Example



Starting with two initial centroids in one cluster of each pair of clusters

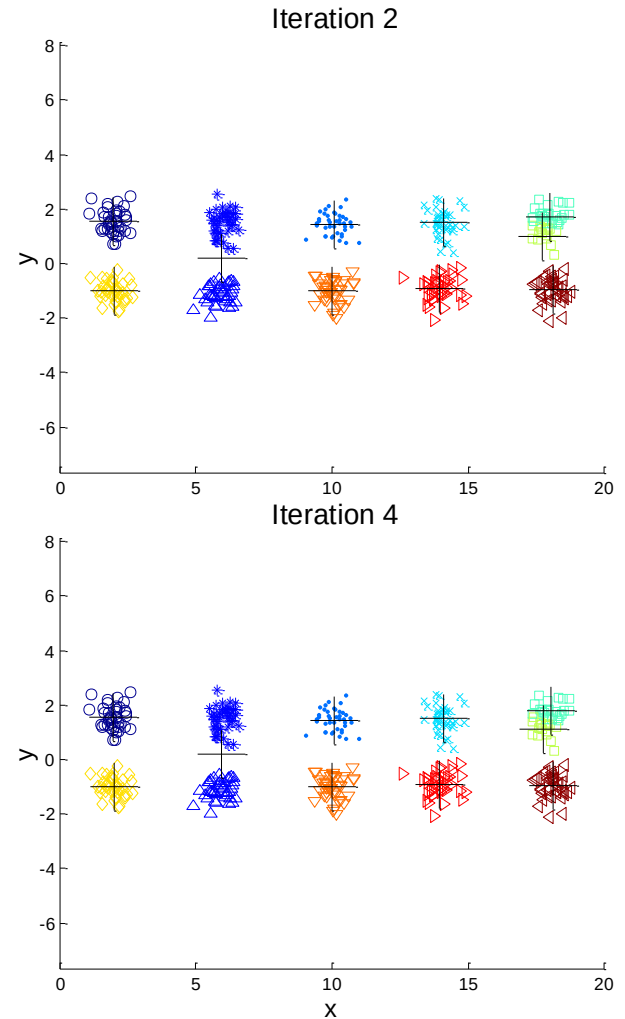
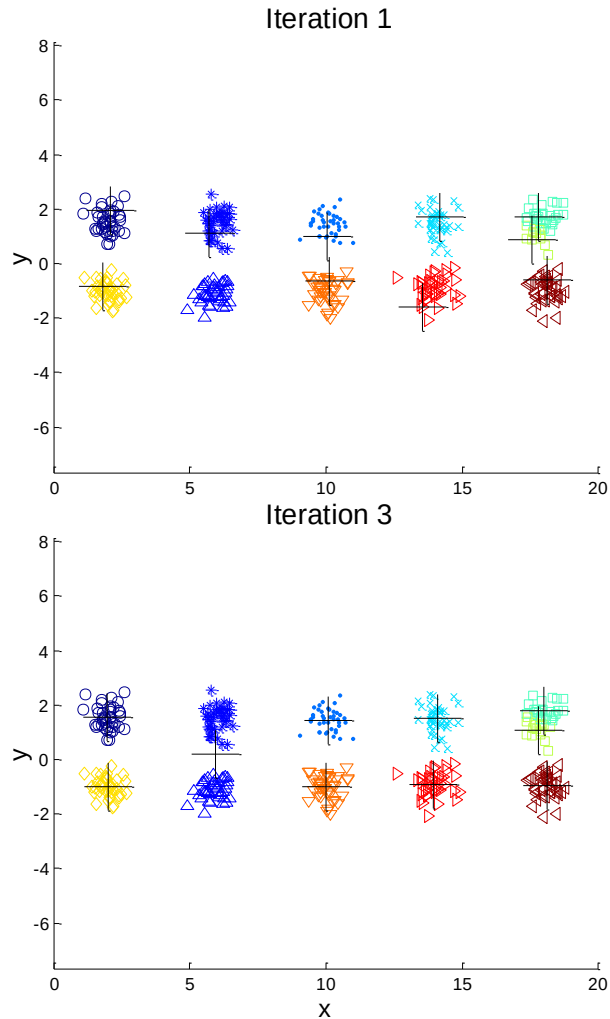
10 Clusters Example

Iteration 4



Starting with some pairs of clusters having three initial centroids, while other have only one.

10 Clusters Example



Starting with some pairs of clusters having three initial centroids, while other have only one.

Solutions to Initial Centroids Problem

- Multiple runs
 - Helps, but probability is not on your side
- Sample and use hierarchical clustering to determine initial centroids
- Select more than k initial centroids and then select among these initial centroids
 - Select most widely separated
- Post-processing
- Bisecting K-means
 - Not as susceptible to initialization issues

Handling Empty Clusters

- Basic K-means algorithm can yield empty clusters
- Several strategies to choose centroid
 - Choose the point that contributes most to SSE (i.e. farthest from the current centroid)
 - Choose a point from the cluster with the highest SSE
 - Typically splits the cluster and reduces overall SSE for clustering
 - If there are several empty clusters, the above can be repeated several times

Updating Centers Incrementally

- In the basic K-means algorithm, centroids are updated after all points are assigned to a centroid
- An alternative is to update the centroids after each assignment (incremental approach)
 - Each assignment updates two or zero centroids (*each point either moves to new cluster or can be in the same cluster*)
 - More expensive
 - Introduces an order dependency
 - Never get an empty cluster
 - Can use “weights” to change the impact

Pre-processing and Post-processing

- Pre-processing
 - Normalize the data
 - Eliminate outliers
- Post-processing
 - Strategies to reduce total SSE
 - Strategies to reduce the number of clusters

-
- Reduce total SSE at the cost of increasing number of clusters
 - Split 'loose' clusters, i.e., clusters with relatively high SSE
 - Introduce new cluster centroid that is farthest from any cluster center
 - Reduce number of clusters while trying to minimize the increase in total SSE
 - Merge clusters that are 'close' and **that have relatively low SSE**
 - Disperse clusters by removing the centroid that corresponds to the cluster and reassigning the points to the other clusters (**select the cluster that increases total SSE the least**)

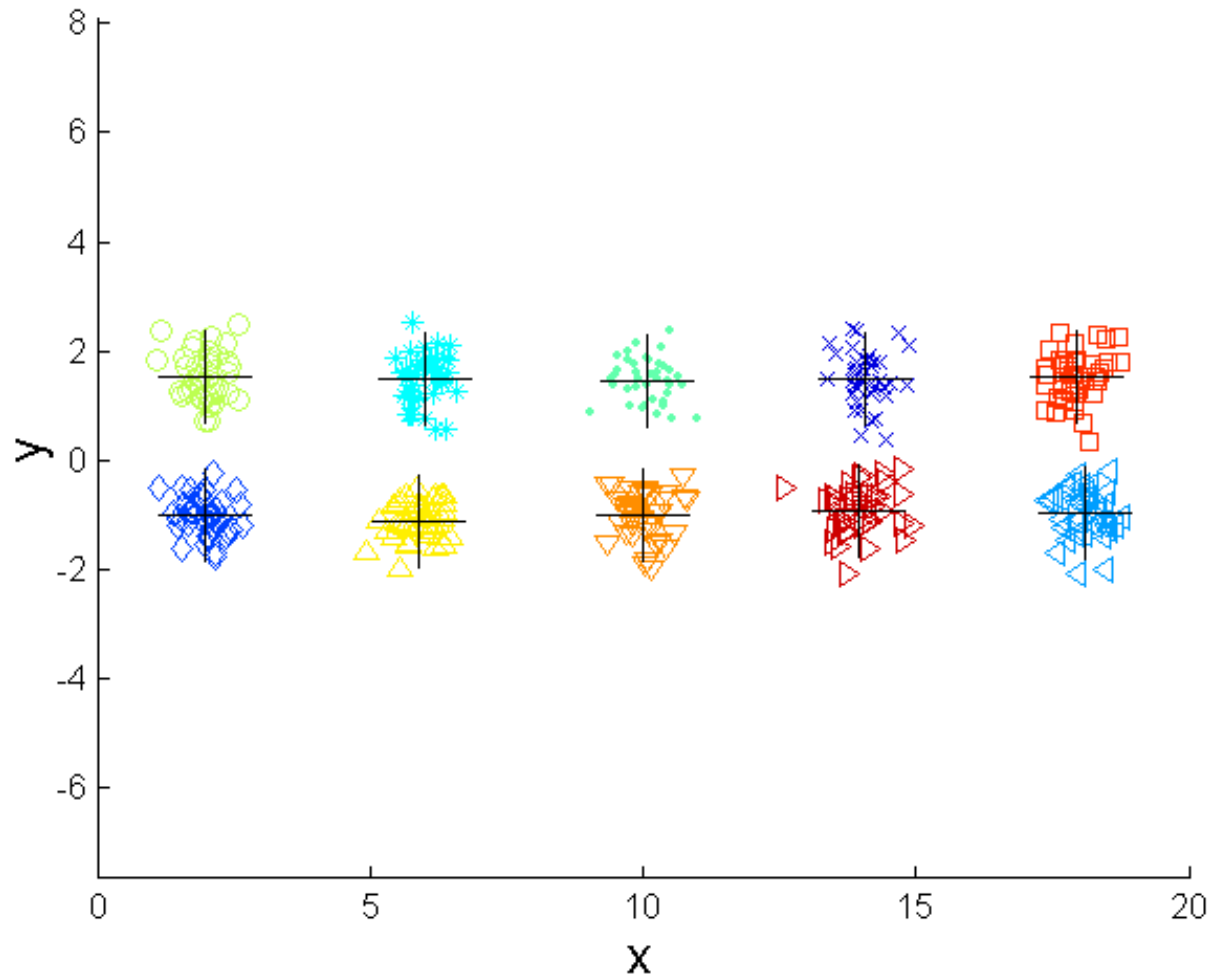
Bisecting K-means

- Bisecting K-means algorithm
 - Variant of K-means that can produce a partitional or a hierarchical clustering

```
1: Initialize the list of clusters to contain the cluster containing all points.  
2: repeat  
3:   Select a cluster from the list of clusters  
4:   for  $i = 1$  to number_of_iterations do  
5:     Bisect the selected cluster using basic K-means  
6:   end for  
7:   Add the two clusters from the bisection with the lowest SSE to the list of clusters.  
8: until Until the list of clusters contains  $K$  clusters
```

Bisecting K-means Example

Iteration 10



Comments on the *K-Means* Method

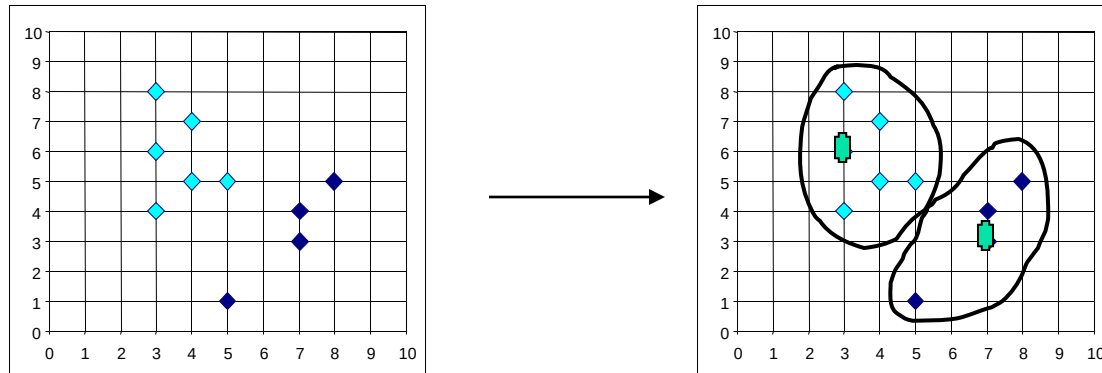
- Strength: *Relatively efficient*: $O(tkn)$, where n is # objects, k is # clusters, and t is # iterations. Normally, $k, t \ll n$.
- Comment: Often terminates at a *local optimum*. The *global optimum* may be found using techniques such as: *deterministic annealing* and *genetic algorithms*
- Weakness
 - Applicable only when *mean* is defined, then what about categorical data?
 - Need to specify k , the *number* of clusters, in advance
 - Unable to handle noisy data and *outliers*
 - Not suitable to discover clusters with *non-convex shapes*

Variations of the *K-Means* Method

- A few variants of the *k-means* which differ in
 - Selection of the initial *k* means
 - Dissimilarity calculations
 - Strategies to calculate cluster means
- Handling categorical data: *k-modes* (Huang'98)
 - Replacing means of clusters with modes
 - Using new dissimilarity measures to deal with categorical objects
 - Using a frequency-based method to update modes of clusters
 - A mixture of categorical and numerical data: *k-prototype* method

What is the problem of the K-means method?

- The k-means algorithm is sensitive to outliers !
 - Since an object with an extremely large value may substantially distort the distribution of the data
- K-Medoids: Instead of taking the **mean** value of the object in a cluster as a reference point, **medoids** can be used, which is the **most centrally located** object in a cluster



An Example (Outlier)

- Consider the 6 points as: 1,2,3,8,9,10,25

K=2:

One partitioning: $\{\{1,2,3\}, \{8,9,10,25\}\}$

Within-cluster variation: $(1-2)^2 + (2-2)^2 + (3-2)^2 + (8-13)^2 + (9-13)^2 + (10-13)^2 + (25-13)^2 = 196$

Second partitioning: $\{\{1,2,3,8\}, \{9,10,25\}\}$

Within-cluster variation:

$$(1-3.5)^2 + (2-3.5)^2 + (3-3.5)^2 + (8-3.5)^2 + (9-14.67)^2 + (10-14.67)^2 + (25-14.67)^2 = 189.67$$

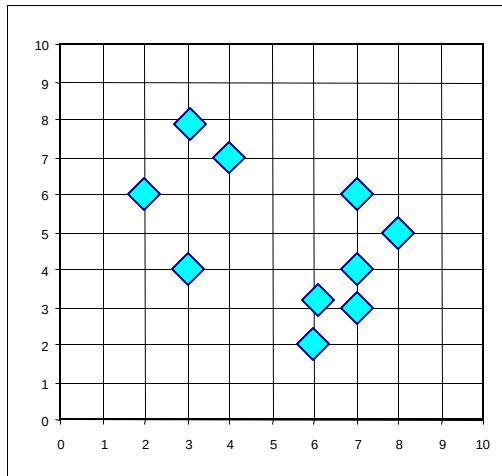
Second cluster is chosen and 8 is assigned to the cluster different from the group that contains 9 and 10 due to the outlier 25. Also the centre 14.67 is far from the other points

The *K-Medoids* Clustering Method

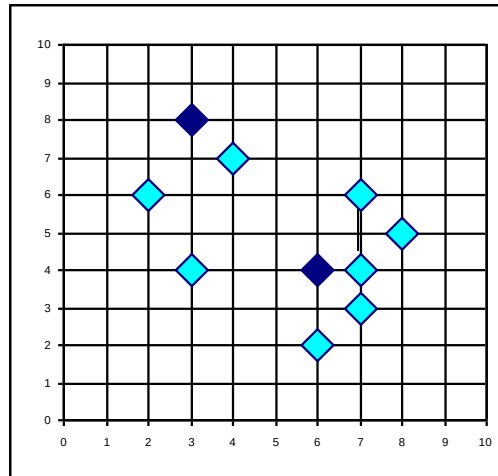
- Find *representative* objects, called medoids, in clusters
- PAM (Partitioning Around Medoids, 1987)
 - starts from an initial set of medoids and iteratively replaces one of the medoids by one of the non-medoids if it improves the total distance of the resulting clustering
 - PAM works effectively for small data sets, but does not scale well for large data sets
- CLARA (Kaufmann & Rousseeuw, 1990)-Clustering with Large Applications
- CLARANS (Ng & Han, 1994): Randomized sampling (Randomized CLARA)

A Typical K-Medoids Algorithm (PAM)

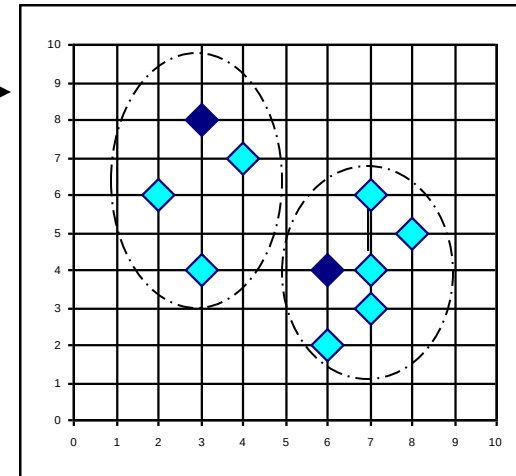
Total Cost = 20



Arbitrary
choose k
object as
initial
medoids



Assign
each
remainin
g object
to
nearest
medoids

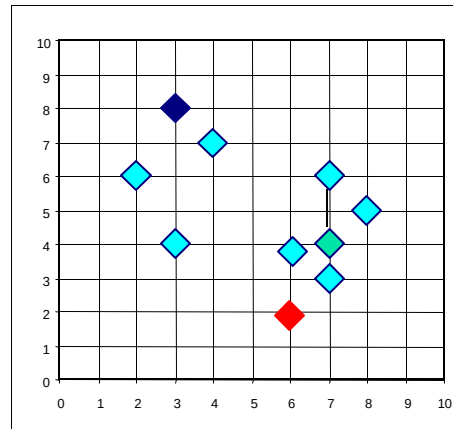


$K=2$

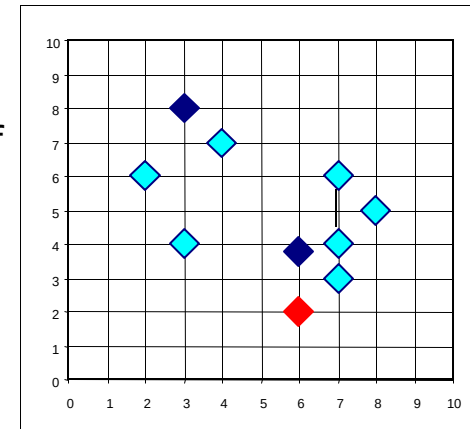
**Do loop
Until no
change**

Swapping O
and O_{random}
If quality is
improved.

Total Cost = 24



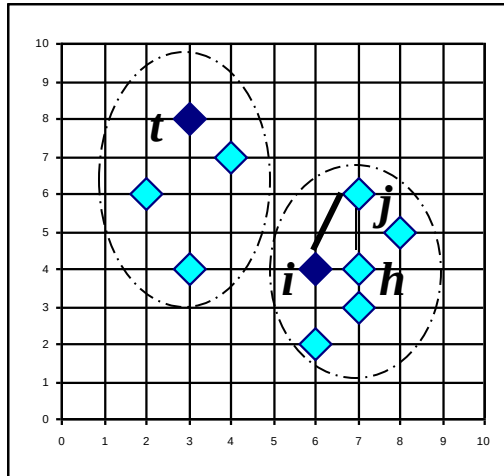
Compute
total cost of
swapping



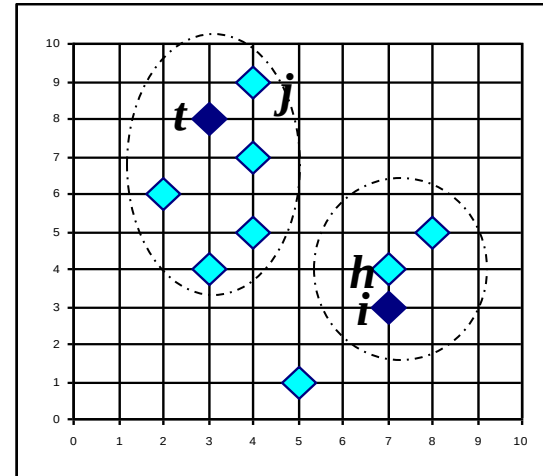
PAM (Partitioning Around Medoids) (1987)

- PAM (Kaufman and Rousseeuw, 1987), built in Splus
- Use real object to represent the cluster
 - Select k representative objects arbitrarily
 - For each pair of non-selected object h and selected object i , calculate the total swapping cost TC_{ih}
 - For each pair of i and h ,
 - If $TC_{ih} < 0$, i is replaced by h
 - Then assign each non-selected object to the most similar representative object
 - repeat steps 2-3 until there is no change

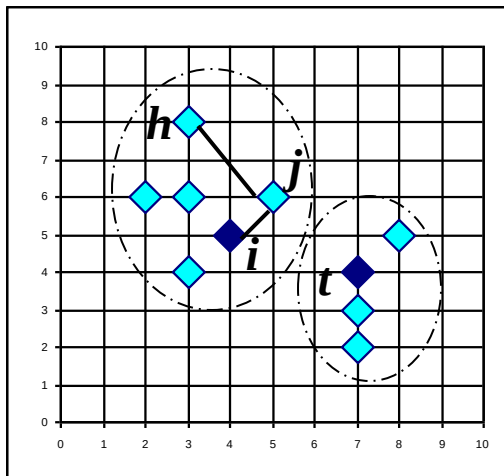
PAM Clustering: Total swapping cost $TC_{ih} = \sum_j C_{jih}$



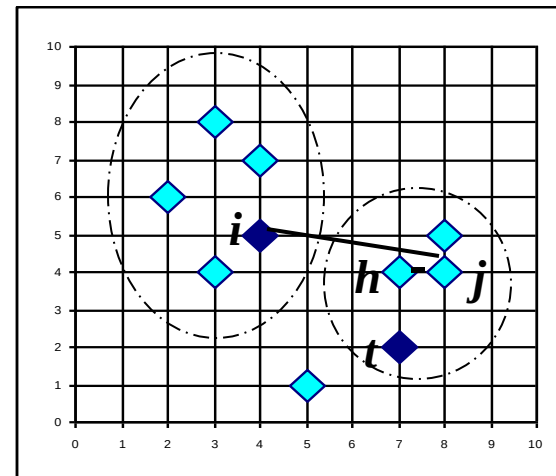
$$C_{jih} = d(j, h) - d(j, i)$$



$$C_{jih} = 0$$



$$C_{jih} = d(j, t) - d(j, i)$$



$$C_{jih} = d(j, h) - d(j, t)$$

What Is the Problem with PAM?

- PAM is more robust than k-means in the presence of noise and outliers because a medoid is less influenced by outliers or other extreme values than a mean
- PAM works efficiently for small data sets but does not **scale well** for large data sets
 - $O(k(n-k)^2)$ for each iteration

where n is # of data-points, k is # of clusters

➔ Sampling based method,

CLARA(Clustering LARge Applications)

CLARA (Clustering Large Applications) (1990)

- CLARA (Kaufmann and Rousseeuw in 1990)
 - Built in statistical analysis packages, such as S+
- It draws *multiple samples* of the data set, applies *PAM* on each sample, and gives the best clustering as the output
- Strength: deals with larger data sets than *PAM*
- Weakness:
 - Efficiency depends on the sample size
 - A good clustering based on samples will not necessarily represent a good clustering of the whole data set if the sample is biased
- Complexity: $O(KS^2 + K(n-K))$